OSS Assessment
Application Security Final Project

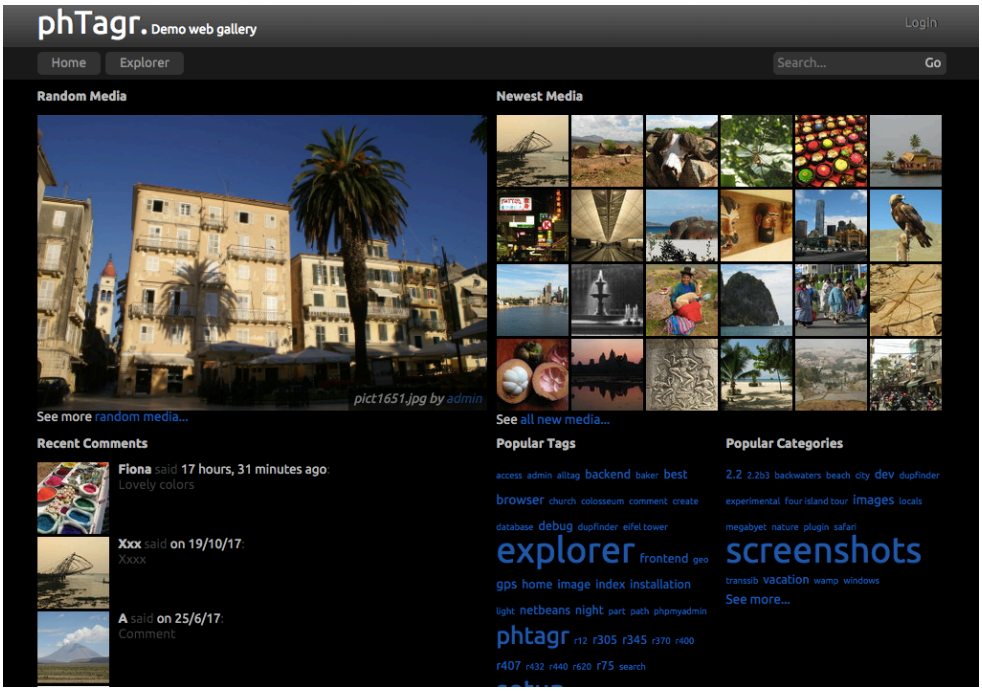
Fiona Sequeira fms299@nyu.edu
Kshitija Deshmukh ksd316@nyu.edu

Summary

As a part of the Application Security Course's final project, we have performed a security assessment on an online photo gallery, phTagr (www.phtagr.org), an open source web gallery to organize your photos with embedded meta data. phTagr allows you to browse your photos and videos by tags and to share your personal digital portfolio with friends.

The purpose of interacting with this application was to use exploitation methods, learnt during this coursework, to evaluate potential vulnerabilities and assess the security related aspects of this web application through a more pragmatic approach.

This report is a detailed analysis of all the code audit that were conducted to inspect security critical functionality and elaborates all the significant findings with suggested remedy solutions.



Description

PhTagr is an open source web gallery written in CakePHP that allows users to organize your photos with embedded meta data. You can browse your own photos and videos by tags and share your digital life with friends. This application allows you to build your own social photo gallery for your community or create your stock photo gallery in your own cloud.
The source code and documentation are available online on GitHub. Thus, you can run PhTagr on your own hardware to have full control about your photos, and videos.

Scope

Due to the size for the overall application only the following framework was considered to be in scope for this assessment -

1. PhTagr Server Core (Includes backed and frontend)

However, Desktop client for PhTagr was considered out of scope for this assessment.

Methodology and Dependencies

We followed the OWASP categorical approach for the application testing. We have taken security tests from checklists on OWASP and closely analyzed if the phtagr application was vulnerable to any of the attack vectors listed out in the OWASP categories.

Since phTagr is now discontinued and all the setup instructions were not in lieu with our OS, we could not find any assistance for plentiful errors and issues during the setup for its latest version available on github(https://github.com/xemle/phtagr), hence to avoid time delay and get started on the application assessment, we continued working on an older version.

A previous version phTagr 2.3, available on their website, had a more straightforward and bug free approach and was used for the security assessment following this.

<u>The environment of the required server had the following dependencies-</u>
Bitnami MAMP Stack – Version 5.6.32
Operating System- Mac OS X
Apache Version- 2.4.29
PHP version- 5.6.31
Browser- Google Chrome

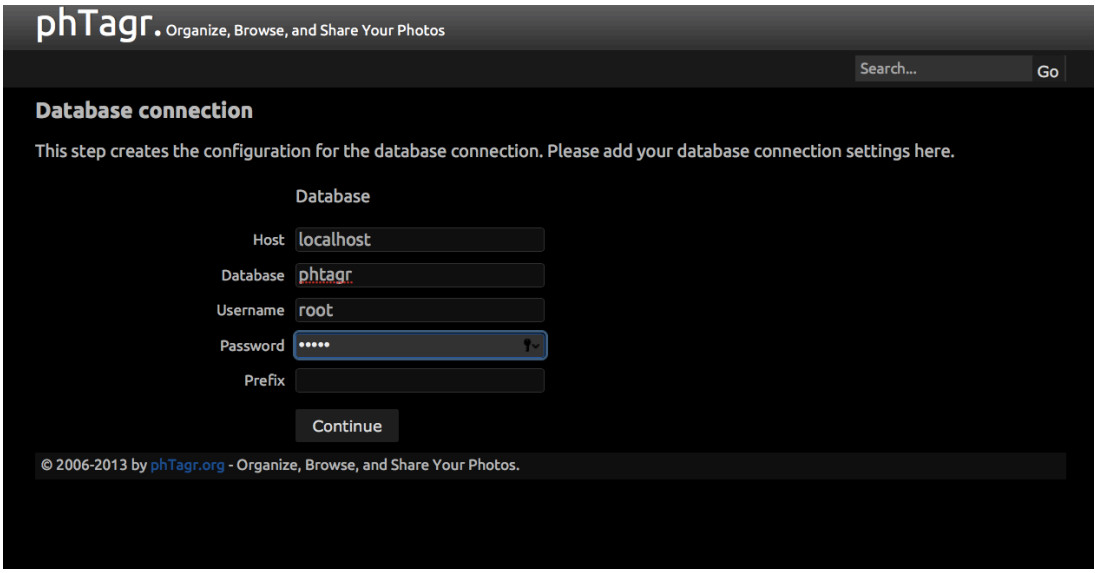
Additional tools used to assess the security analysis-

OWASP ZAP 2.7.0 - A tool that can helps in automatically finding security vulnerabilities in web applications while developing and testing.

Kindly note that we have performed all our analysis on the application after we had successfully done the download and installation instruction redirected on [this link](). Some of the bugs come across here in this report can be resolved by modifying the settings in the application configuration.

The setup page requires you to setup the database connection, which is the same credentials entered during the MAMP setup and can also be connected to the localhost at the database server via this localhost:PORT/phpmyadmin

Here is a snapshot of the localhost:PORT/phtagr/setup after installation-



Application Threat Model: phTagr

Application Threat Model is a structured method of analyzing the security of website/application and can be seen as a complementary process to secure development. ATM requires analysis of the security threats from **viewpoint** of an **external** attacker rather than a developer and finding the weak points for attacking the system.

The first step includes **decomposing the website** and creating a **Data Flow Diagram** in order to understand the flow process within the website. The next step involves the identification of all the threats associated with each aspect of the flow process. The final step includes listing the

security measures that have been taken by the website owner as well as the measures to mitigate the risks which have not been dealt with.

Threat Model Information	
Application Name	phTagr
Application Version	2.3
Description	phTagr is a web based photo managing and sharing application. It has a wide range of features including image editing using filters, adding metadata such as captions, geotagging, local and cloud storage and flexible access management.
Document Owner	Fiona Sequeira, Kshitija Deshmukh
Participants	Fiona Sequeira, Kshitija Deshmukh
Reviewer	Prof. Kelly Lum

External Dependencies	
ID	Description
1	The application website runs on a Linux server running Apache.
2	The database server will be MySQL and it will run on a Linux server.
3	The connection between the application server and the database server will be over a private network.
4	The Web server is behind a firewall but it can be accessed without TLS.

Entry Points			
ID	Name	Description	Trust Levels
1	HTTPS Port	The website is accessible even without TLS. All the webpages accept insecure HTTP connections.	1. Anonymous Web User 2. Users with invalid credentials 3. Users with Valid Credentials 4. Web Administrator

1.1	Home Page	The home page is the entry point for all the users and admin.	1. Anonymous Web User 2. Users with invalid credentials 3. Users with Valid Credentials 4. Web Administrator
1.2	Login Page	All the users and admin require logging in to access the personal gallery. However, the explorer can be accessed without logging in.	1. Anonymous Web User 2. Users with invalid credentials 3. Users with Valid Credentials 4. Web Administrator
1.2.1	Login Function	The login function accepts user supplied credentials and compares them to those in the database.	1. Anonymous Web User 2. Users with Valid Credentials 3. Web Administrator
1.3	Explorer Page	The explorer displays the recently uploaded or edited images and is also accessible to all users.	1. Anonymous Web User 2. Users with invalid credentials 3. Users with Valid Credentials 4. Web Administrator
1.4	My gallery	The page displays the user's personal images	1. Users with Valid Credentials 2. Web Administrator

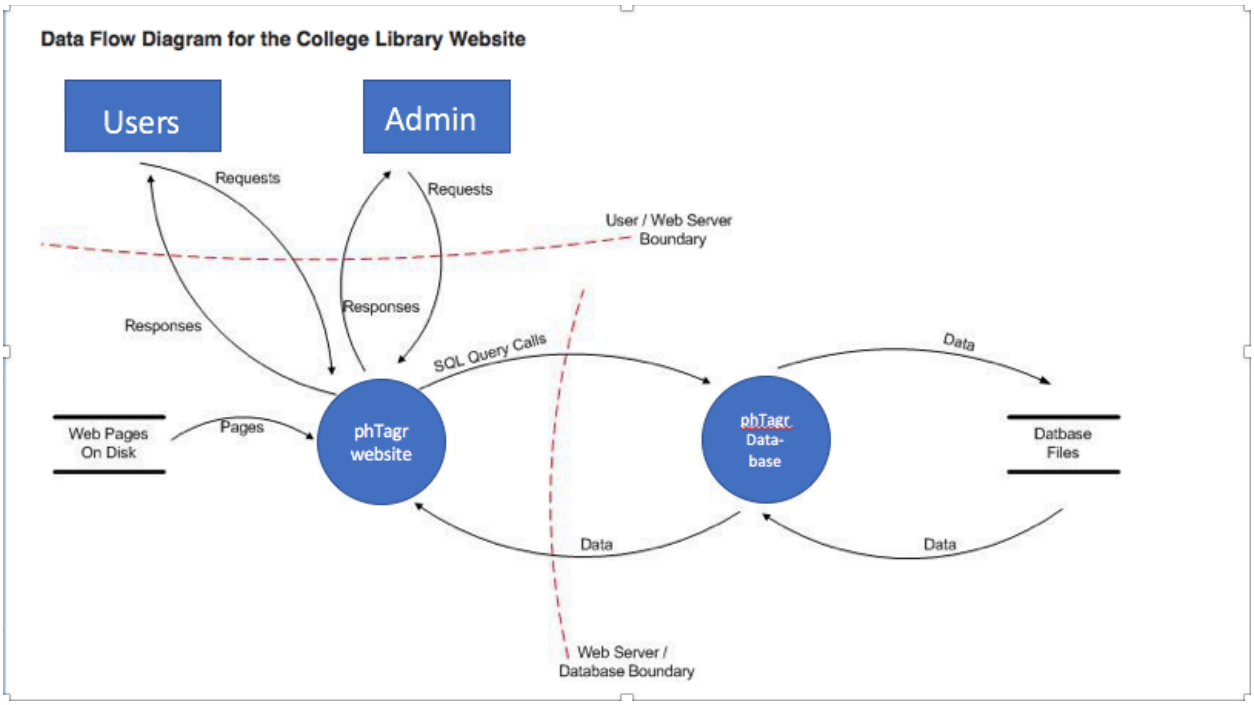
Assets			
ID	Name	Description	Trust Levels
1	Application Users, Anonymous Users.	Assets related to users and visitors.	
1.1	User Login Details	The login credentials that a user or administrator will use to login into the website.	1. User with valid credentials

			2. Database server administrator 3. Website administrator 4. Database read administrator 5. Database read/write administrator
1.2	Personal Data	The website will store the images uploaded by the users and the metadata.	1. Database server administrator 2. Website administrator 3. Database read administrator 4. Database read/write administrator
2	System	Assets relating to the underlying system	
2.1	Availability of the application website	The website should be available 24 hours a day and accessible to all the visitors and users.	1. Database server administrator 2. Website administrator
2.2	Ability to upload and search images and edit metadata	This is the ability to upload images and search for them. The ability to edit the image should be limited to the owner of the image.	1. User with Valid Credentials 2. Database server administrator 3. Website administrator 4. Database read/write administrator
3	Website	Assets related to the application website	
3.1	Login Session	This is the login session of the user.	1. User with valid login 2. Website administrator
3.2	Access to Server	Access to Database server allowing to the administer the database and modify all the data contained within.	1. Database Server Administrator
3.3	Ability to create user	The ability to create users either normal or admin.	1. Website administrator

3.4	Access to Audit Data	The ability to audit the data on the website including all the image and metadata uploaded as well as the website itself.	1. Website Administrator 2. Database read/write administrator

Trust Levels		
ID	Name	Description
1	Anonymous Web User	A user who is browsing through the website but has not logged in.
2	User with Invalid Login Credentials	A user who has logged into the website.
3	User with Valid Credentials	A user who is trying to log into the website with invalid credentials.
4	Database Server Administrator	The database server administrator has read and write access to the database used by the application website.
5	Website Administrator	The website administrator can configure the application website.
6	Database Read User	Database user account with read access only.
7	Database Read/Write User	Database user with read/write access.

Data Flow Diagrams



Threat Categorization

The first step in the determination of threats is threat categorization such as STRIDE. A threat list has been described below:

STRIDE Threat List		
Type	Examples	Security Control
Spoofing	Illegally accessing another user's credentials, such as username and password	Authentication
Tampering	Maliciously modify persistent data or data in transit to the website	Integrity
Repudiation	Perform illegal operations in a system unable to trace the prohibited activity	Non-repudiation
Information Disclosure	Read a file with unauthorized access	Confidentiality
Denial of Service	Deny access to users by temporary unavailability of website	Availability
Elevation of Privilege	Gain unauthorized access or compromise the system	Authorization

Understanding the Code Structure

This application uses CakePHP which is an open source web framework. It follows the model-view-controller(MVC) approach and is written in PHP.

Model–view–controller (MVC) is a software architectural pattern that divides a given application into three interconnected parts.

This is done to separate internal representations of information from the ways information is presented to, and accepted from, the user. The MVC design pattern disassociates these major components adding room for efficient code reuse and parallel development.

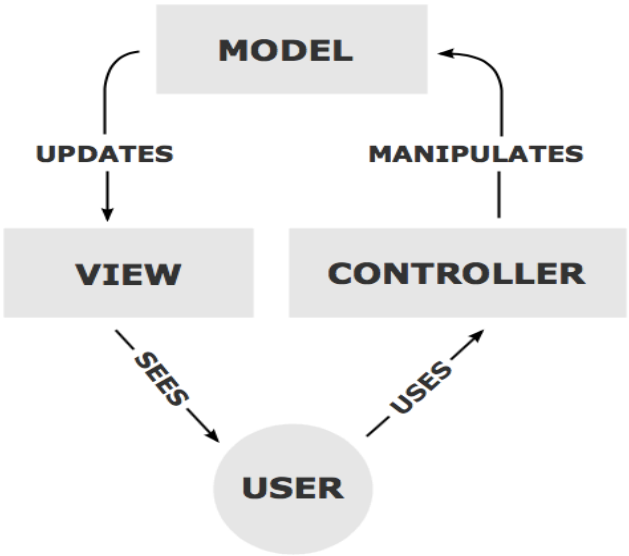
Components

- The **model** is the central component of the pattern. It expresses the application's behavior in terms of the problem domain, independent of the UI and it directly manages the data, logic and rules of the application.
- A **view** could be any output representation of information, such as a chart or a diagram and mostly handles the front end. Multiple views of the same information are possible, such as a bar chart for management and a tabular view for accountants.
- The **controller**, accepts input and converts it to commands for the model or view, essentially for each aspect of the application there would be a controller.

Interactions

The MVC design defines the interactions between the three components -

- A model stores data that is retrieved according to commands from the controller and displayed in the view.
- A view generates new output to the user based on changes in the model.
- A controller can send commands to the model to update the model's state (e.g., editing a document). It can also send commands to its associated view to change the view's presentation of the model (e.g., scrolling through a document, movement of document)



We have mainly analyzed the MVC structure of phTagr for the security assessment, which have the following components

The main components of the webpage is a personal digital image portfolio with user sign up page, upload page and explorer page. However since there isn't an option to create a new user, we are limited to testing the functionality on this scale.

Models used
AppModel.php, Comment.php, Field.php, Group.php, GroupsMedia.php, GroupsUser.php, Guest.php, Lock.php, Media.php, MyFile.php, Option.php, Property.php, User.php

Views used
Browser, Comments, Elements, Emails, Explorer, Groups, Guests, Helper, Images, Layouts, Options, Setup, System, Themed/Mobile, Users, Webdav, home

Controller used
AppController.php,BrowserController.php, CommentsController.php, ExplorerController.php, GroupsController.php, GuestsController.php, HomeController.php, ImagesController.php, LocationController.php, MediaController.php, OptionsController.php, SetupController.php, SystemController.php, UsersController.php, WebdavController.php

APPLICATION THREAT MODEL

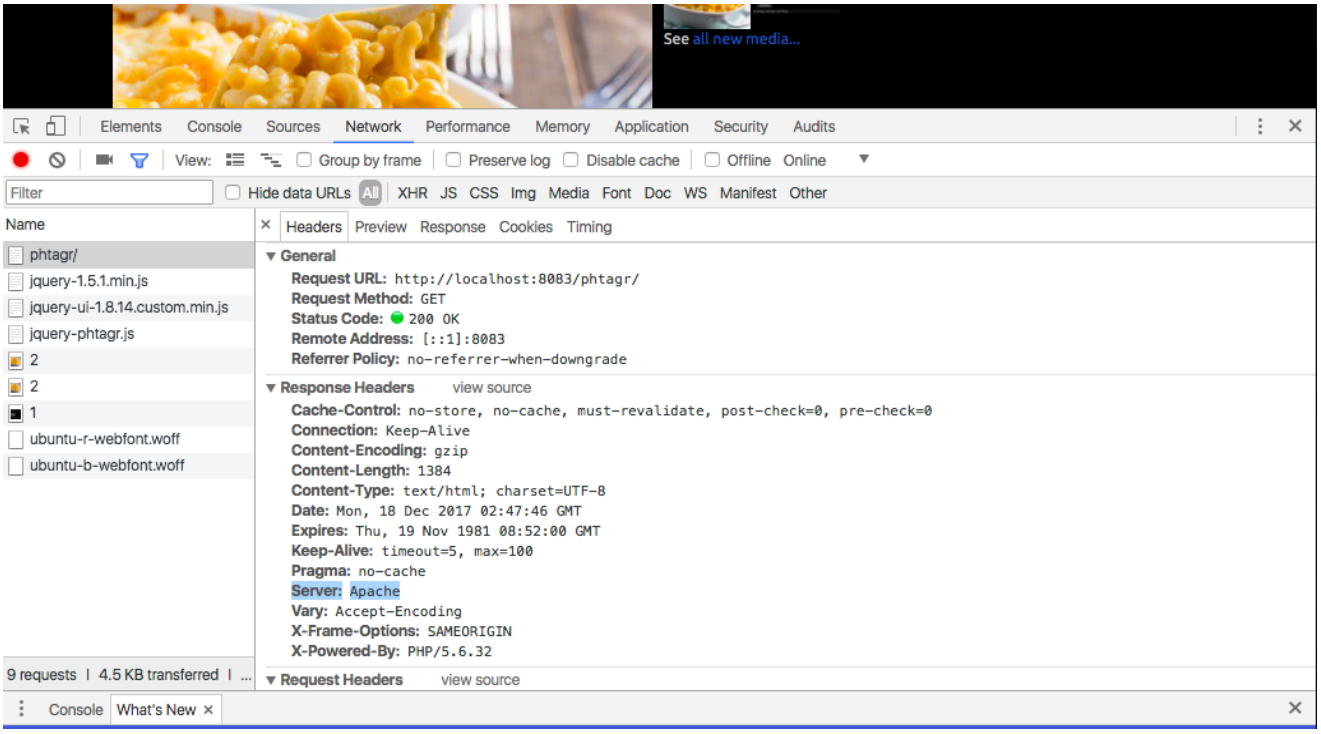
Potential threats on the application vs Mitigation strategies for the above if any
-user can upload a virus, file is not being checked

Testing Strategies and Results

Information Gathering

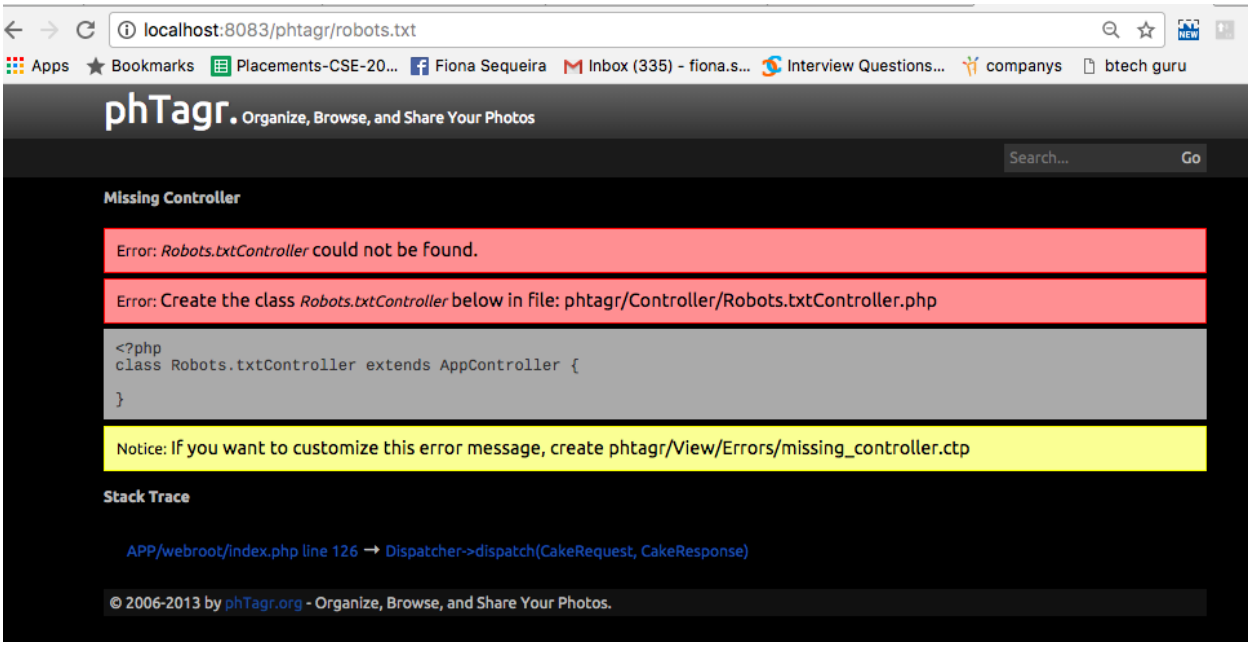
Security Test	OWASP Category	Yes/No	Results
Fingerprint web server	OTG-INFO-002	Yes	Able to fingerprint web server from response headers.

Explanation:
Since there are a several number of different versions of webservers, it is really important for the penetration tester to know the type of webserver that is being tested. This information can be derived by sending specific commands and analyzing the output as each webserver reacts differently to these commands.



Security Test	OWASP Category	Yes/No	Results
Review Webserver Metafiles for information leakage	OTG-INFO-003	Yes	Web server does not contain robots.txt file

Explanation:
We check for 'robots.txt' to test for information leakage of the web application's directory or folder path. The robots.txt is a simple txt file placed on your web server which tells web crawlers if they should access a file or not. So, we deploy our application and start the server. Access the application using 'localhost: 8083/phtagr/robots.txt' as mentioned in the screenshot below.



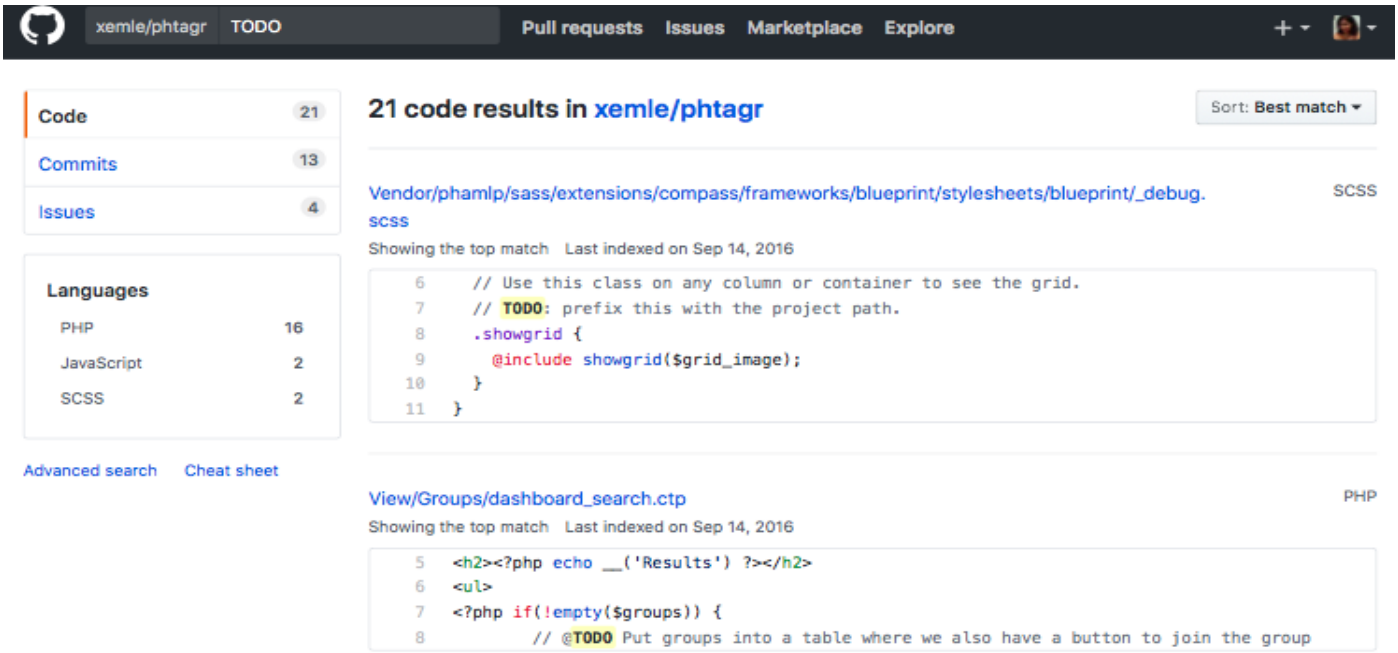
Thus, the application web server doesn't contain the robots.txt file.

Security Test	OWASP Category	Yes/No	Results
Enumerate Applications on Webserver	OTG-INFO-004	Not Applicable	Application is hosted locally for assessment

Explanation:
Currently the web server only hosts one application. Therefore, this test is not valid.

Security Test	OWASP Category	Yes/No	Results
Review Webpage comments and metadata for information Leakage	OTG-INFO-005	FAIL	Several TODO comments exist in the source code which leads to the leakage of information.

Explanation:
The application source code present on GitHub has comments with a tag 'TODO'. These comments highlight the problems in the application those need to be fixed. This can be a threat as it can reveal sensitive information that can help the attacker gain more insight about the application.



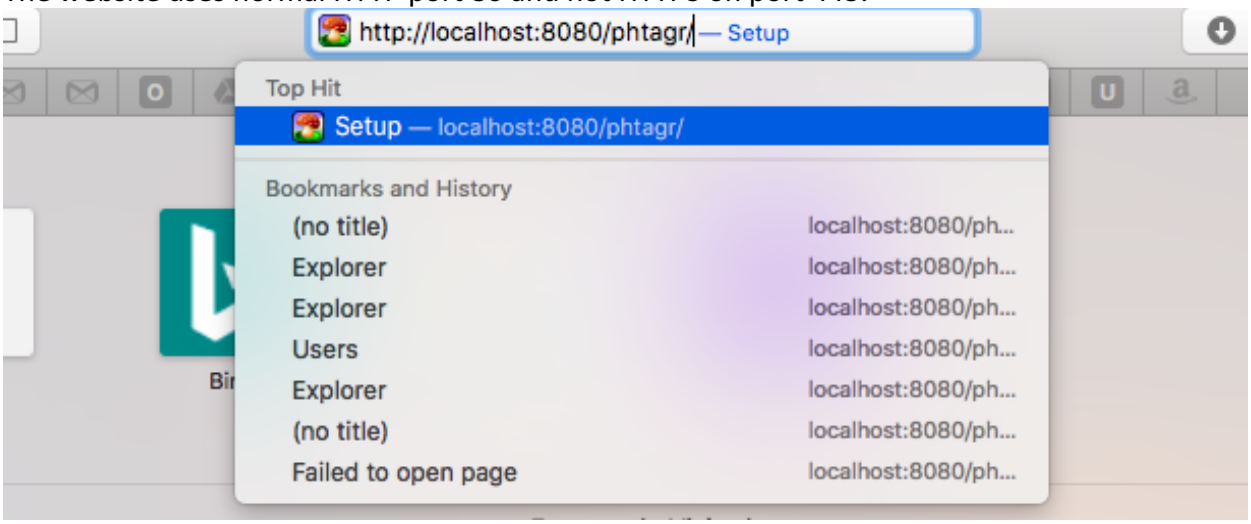
Configuration and Deployment Management Testing

Security Test	OWASP Category	Yes/No	Results
Review Old, Backup and Unreferenced files for Sensitive information	OTG-CONFIG-004	PASS	No sensitive information present.

Explanation:
There are few files present in the application such as .htaccess which stores the old and unreferenced data but they do not contain any sensitive information.

Security Test	OWASP Category	Yes/No	Results
Test HTTP Strict Transport Security	OTG-CONFIG-007	Not Applicable	HTTPS is not used

Explanation
The website uses normal HTTP port 80 and not HTTPS on port 443.



Authentication Testing

Security Test	OWASP Category	Yes/No	Results
Testing for Credentials Transported over an Encrypted Channel	OTG-AUTHN-001	Not Applicable	HTTPS is not used on this application

Explanation

Testing for credentials transport means verifying that the user's authentication data are transferred via an encrypted channel to avoid being intercepted by malicious users. This analysis revolves around trying to understand if the data travels unencrypted from the web browser to the server, or if the web application takes the appropriate security measures using a protocol like HTTPS. The HTTPS protocol is built on TLS/SSL to encrypt the data that is transmitted which is not the case in the phTagr that uses HTTP.

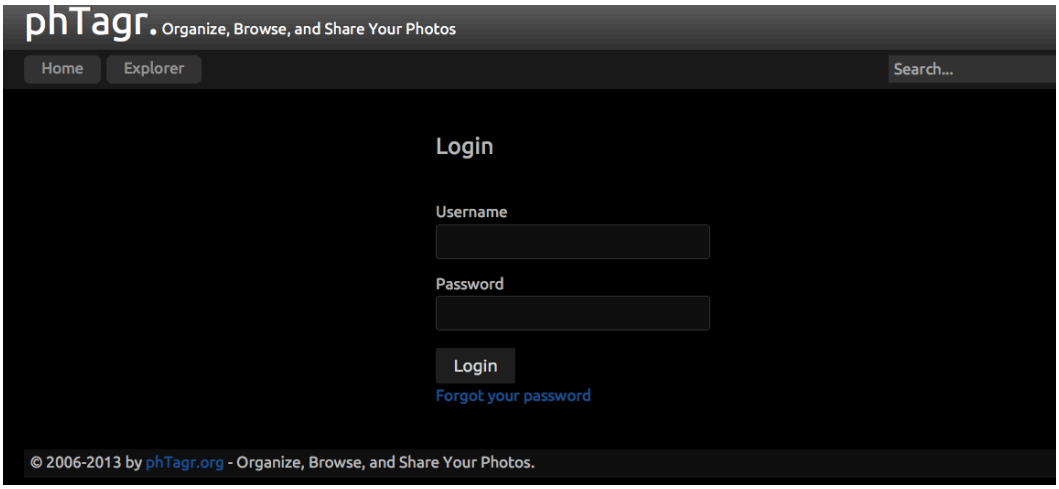
Security Test	OWASP Category	Yes/No	Results
Testing for Weak lock out mechanism	OTG-AUTHN-003	FAIL	No Lockout Policy Enforced. Login Form Is Brute Forcible.

Explanation
Account lockout mechanisms are used to mitigate brute force password guessing attacks. Accounts are typically locked after 3 to 5 unsuccessful login attempts and can only be unlocked after a predetermined period of time. However, in the case of phTagr there is no lockout mechanism implemented, and a user is allowed multiple number of retries.

Suggested Remedy
User should have a stricter password policy implemented, with security questions/answers and multi factor authentication.

Security Test	OWASP Category	Yes/No	Results
Testing for Vulnerable Remember Password	OTG-AUTHN-005	Not Applicable	No remember password option on the log in page.

Explanation
Having the browser store passwords is not only a convenience for end-users, but also for an attacker. If an attacker can gain access to the victim's browser (e.g. through a Cross Site Scripting attack, or through a shared computer), then they can retrieve the stored passwords. As this option is not present on the phTagr website this issue is not applicable.



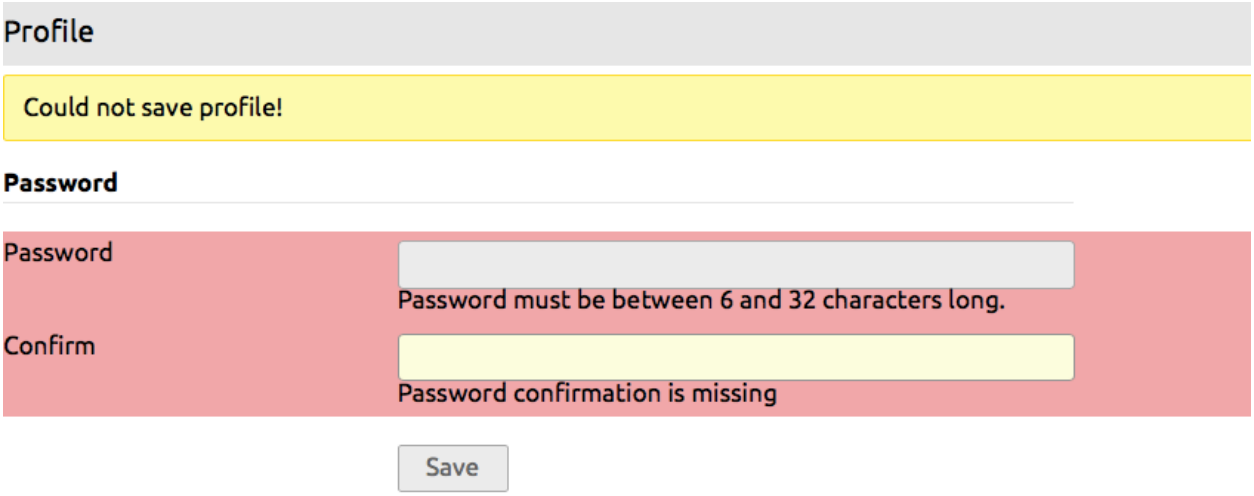
Security Test	OWASP Category	Yes/No	Results
Testing for Weak password policy	OTG-AUTHN-007	FAIL	.htaccess is well implemented.

Explanation

The most prevalent and most easily administered authentication mechanism is a static password. The password represents the keys to the application, but is often subverted by users in the name of usability. In phtagr there is no such checks for password and the weakest of passwords like 123456 can easily be registered on the system.

Suggested Remedy

User should have a strict password policy implemented, with requirements of the password containing alphanumeric characters as well as special characters, along with security questions/answers and multi factor authentication.



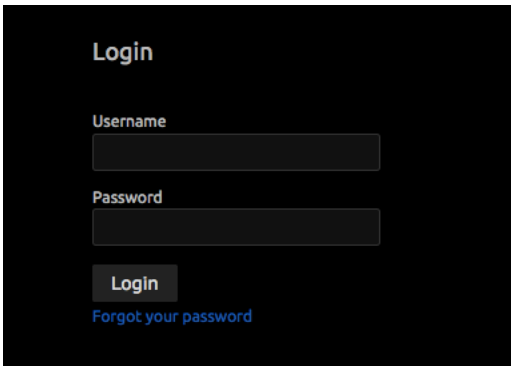
Security Test	OWASP Category	Yes/No	Results
Testing for Weak security question/answer	OTG-AUTHN-008	FAIL	No security question/answer present on the user creation page

Explanation

Often called "secret" questions and answers, security questions and answers are often used to recover forgotten passwords, or as extra security on top of the password. They are typically generated upon account creation and require the user to select from some pre-generated questions and supply an appropriate answer.

Suggested Remedy

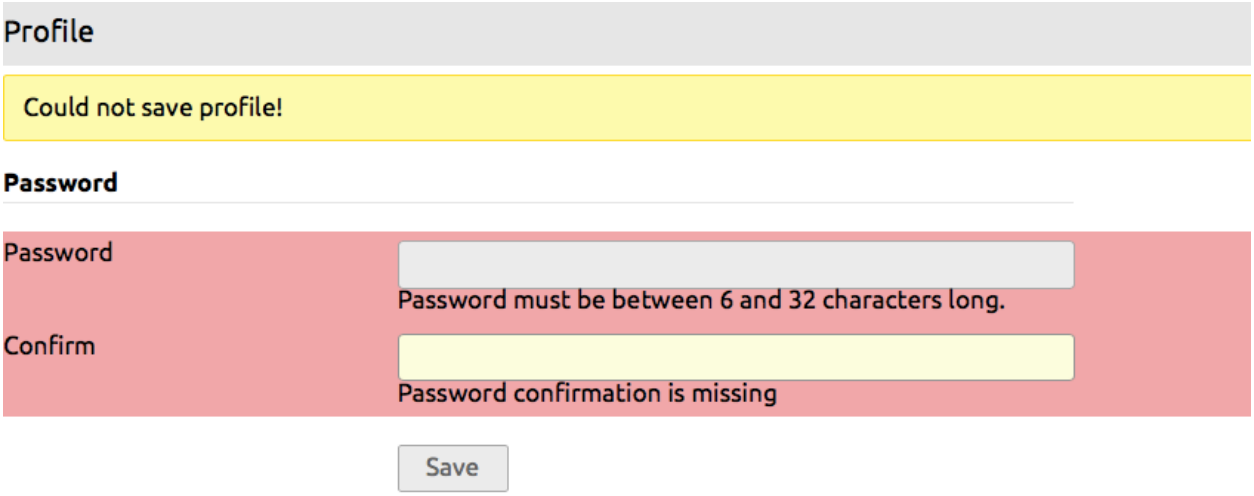
User should have an option of security question and answer to be setup and linked with his profile to ensure security of the application and better secureness of the user.



Security Test	OWASP Category	Yes/No	Results
Testing for weak password change or reset function	OTG-AUTHN-009	FAIL	Password change doesn't require

Explanation
They are typically generated upon account creation and require the user to select from some pre-generated questions and supply an appropriate answer. They may allow the user to generate their own question and answer pairs.

Suggested Remedy
User should have an option of secure password to be setup and the password should be a combination of alphanumeric and special characters as per the standard of strong passwords.



Authorization Testing

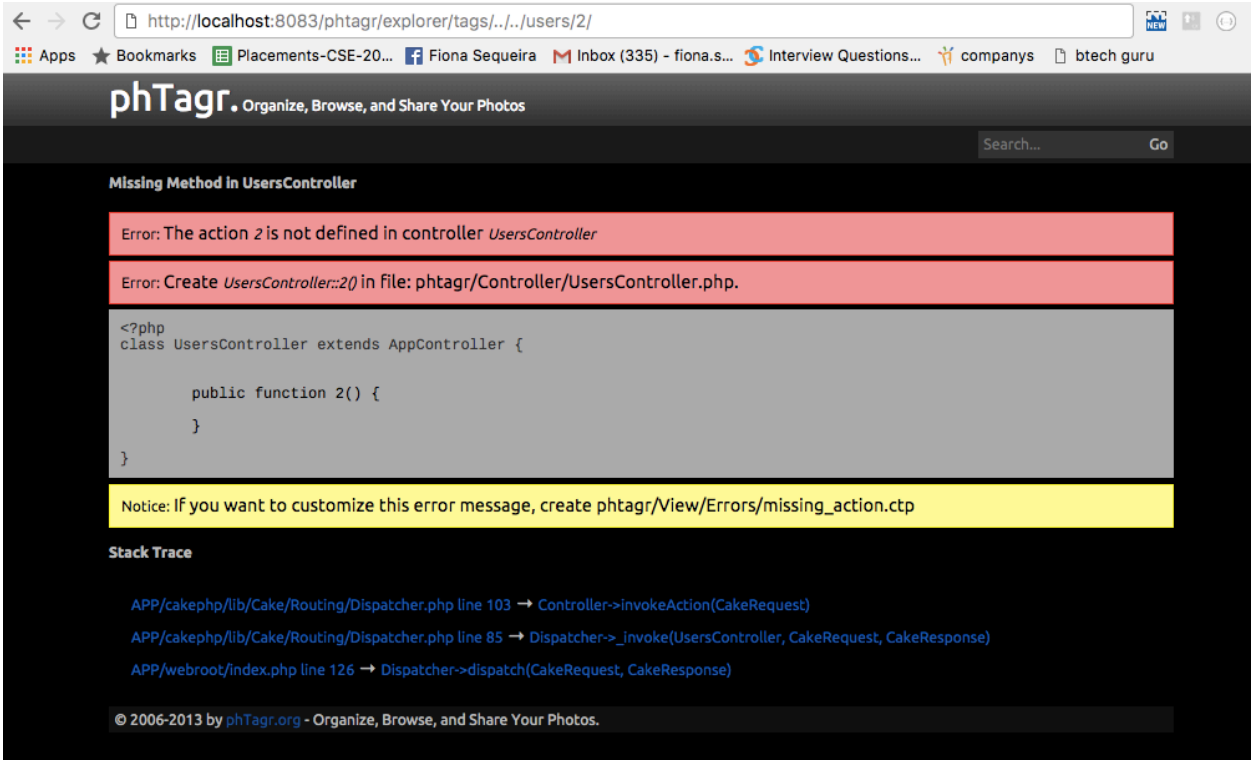
Security Test	OWASP Category	Yes/No	Results
Testing Directory traversal/file include	OTG-AUTHZ-001	PASS	.htaccess is well implemented.

Explanation:
Web applications implement authentication mechanisms to control access to files and resources. Web servers try to confine users' files inside a "root directory" or "web document root", which represents a physical directory on the file system. Users have to consider this directory as the base directory into the hierarchical structure of the web application.

In this test we have tried to use the dot-dot-slash attack(../) or directory traversal but we were not allowed to do so. Hence this test has been **passed**.

.htaccess

```
<IfModule mod_rewrite.c>
    RewriteEngine on
    RewriteRule    ^$    webroot/    [L]
    RewriteRule    (.*) webroot/$1    [L]
</IfModule>
```



Security Test	OWASP Category	Yes/No	Results
Testing for Insecure Direct Object References	OTG-AUTHZ-004	PASS	Direct Object References are prohibited

Explanation

Some application provides direct access to objects based on user-supplied input and these vulnerabilities provide a potential for attackers to bypass authorization and access resources in the system directly, for example database records or files.

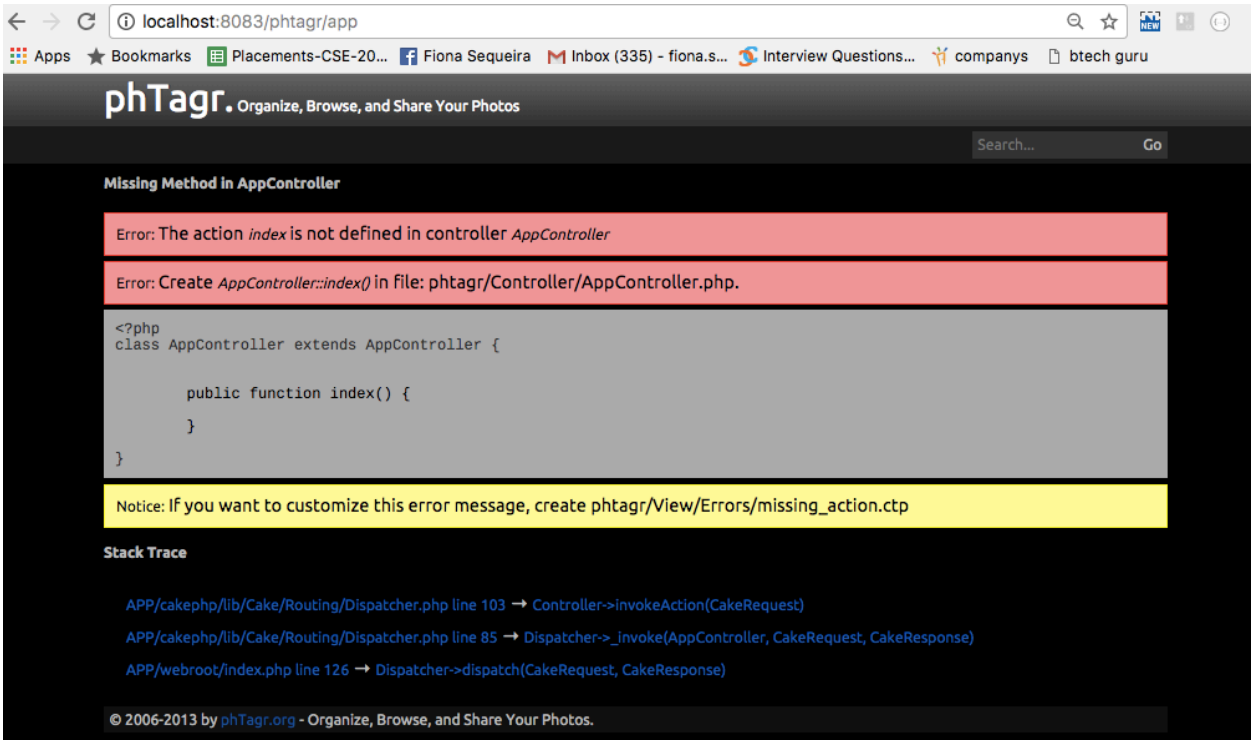
In phTagr, every object or dependency reference is appended with a secure session key, which makes session identification by the application easy. Images are not directly referenced, but redirected. However, we have information leakage present including stack traces, which we will go more in detail in the further vulnerability tests under OWASP Category Testing for Error Handling.

Code Snippet of a session checker from View's Element; topnav.ctp

```
<?php
  if ($this->Session->check('User.id')) {
    $userId = $this->Session->read('User.id');
    $role = $this->Session->read('User.role');
    $name = $this->Session->read('User.username');
    if ($role >= ROLE_SYSOP) {
```

Code Snipper of startSession() from Controller's Component : fast_file_responder.php

```
function startSession() {
  if (!isset($_COOKIE[$this->sessionCookie])) {
    return;
  }
  session_id($_COOKIE[$this->sessionCookie]);
  session_name($this->sessionCookie);
  session_start();
  if (isset($_SESSION[$this->sessionKey])) {
    $this->items = (array) $_SESSION[$this->sessionKey]['items'];
    $this->deleteExpiredItems();
  }
}
```

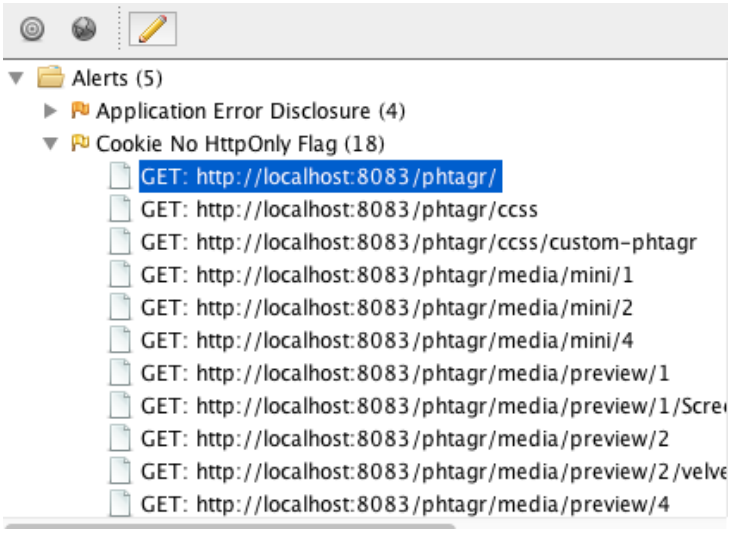


Session Management Testing

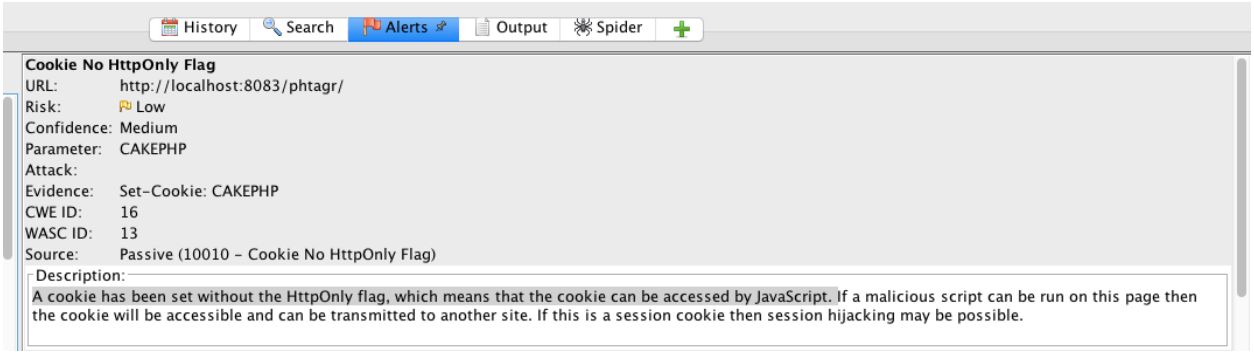
Security Test	OWASP Category	Yes/No	Results
Testing for Cookies attributes	OTG-SESS-002	FAIL	A cookie has been set without the HttpOnly flag

Explanation:
The importance of secure use of Cookies cannot be understated, especially within dynamic web applications, which need to maintain state across a stateless protocol such as HTTP. To understand the importance of cookies it is imperative to understand what they are primarily used for. We have analyzed this section via the OWASP ZAP tool and we achieved the following alert which signifies A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript.

Remedy:
Use the HTTPOnly flag while setting cookies.




```
HTTP/1.1 200 OK
Date: Tue, 19 Dec 2017 01:31:12 GMT
Server: Apache
X-Frame-Options: SAMEORIGIN
X-Powered-By: PHP/5.6.32
Set-Cookie: CAKEPHP=hsrd9pepfhsg7eb2n22h38vhv6; expires=Tue, 19-Dec-2017 05:31:12 GMT; Max-Age=14400; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 4367
```

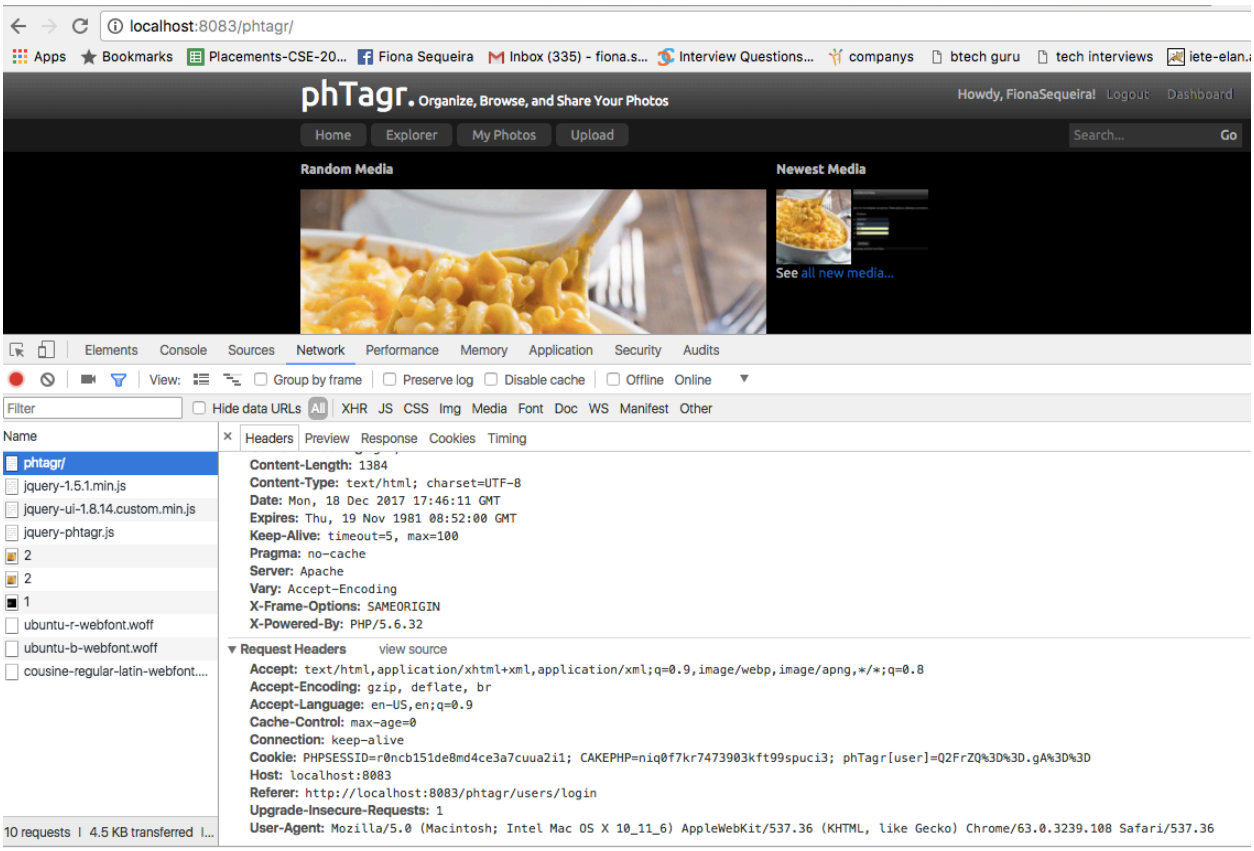


Security Test	OWASP Category	Yes/No	Results
Testing for Exposed Session Variables	OTG-SESS-004	FAIL	[user] tagged variable explicitly passed in a session

Explanation:
The Session Tokens (Cookie, SessionID, Hidden Field), if exposed, will usually enable an attacker to impersonate a victim and access the application illegitimately. It is important that they are protected from eavesdropping.

The following Console snapshot was taken after a successful log-in wherein the network layer explicitly indicates a user tagged variable passed. Although the username is hashed it becomes available for any attacker particularly whilst in transit between the client browser and the application servers.

Suggested Remedy, instead of using the tag 'user' and label it differently to ensure protection.



Security Test	OWASP Category	Yes/No	Results
Testing for Logout functionality	OTG-SESS-006	PASS	User is not allowed to traverse after Logging out

Explanation:

There are multiple issues which can prevent the effective termination of a session. For the ideal secure web application, a user should be able to terminate at any time through the user interface. Every page should contain a log out button on a place where it is directly visible. Pages that were opened during an active session must log the user out from all sessions once user has hit the Log out option. This test has **PASSED**

Security Test	OWASP Category	Yes/No	Results
Testing for Session Timeout	OTG-SESS-007	FAIL	No session timeout functionality

Explanation:
As per the OTG Session description for this issue all applications should implement an idle or inactivity timeout for sessions. This timeout defines the amount of time a session will remain active in case there is no activity by the user, closing and invalidating the session upon the defined idle period since the last HTTP request received by the web application for a given session ID. We experimented keeping a session active for over a day but there was never a time out.

This security test has **FAILED**

Suggested Remedy:
Code snippet from Config's core.php shows that Session.timeout is discussed but is not setup and not enforced in default configuration of phTagr. Setting up this parameter should be enforced during phTagr setup.

Config/core.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
230  * - `Session.cookie` - The name of the cookie to use. Defaults to 'CAKEPHP'
231  * - `Session.timeout` - The number of minutes you want sessions to live for. This
     timeout is handled by CakePHP
232  * - `Session.cookieTimeout` - The number of minutes you want session cookies to live
     for.
```

Input Validation Testing

Security Test	OWASP Category	Yes/No	Results
Testing for Reflected Cross Site Scripting	OTG-INPVAL-001	PASS	Reflected Cross Site Scripting is handled

Explanation:
This occurs when an attacker injects browser executable code within a single HTTP response. The attack string is included as part of the crafted URI or HTTP parameters, improperly processed by the application, and returned to the victim.
This test has PASSED.

The below screenshots verify the security-test conducted.

Code snippet to sanitize Controller's CommentsController.php

Controller/CommentsController.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

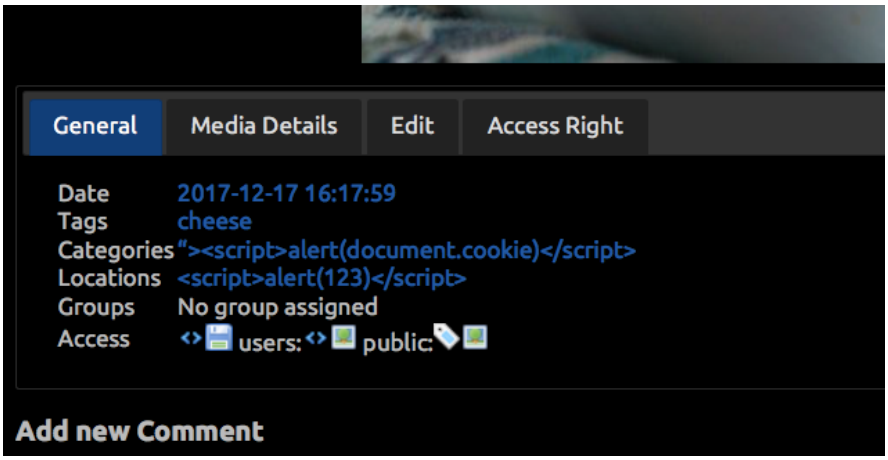
```
15  * @license       GPL-2.0 (http://www.opensource.org/licenses/GPL-2.0)
16  */
17
18 App::uses('Sanitize', 'Utility');
19 App::uses('CakeEmail', 'Network/Email');
20
21 class CommentsController extends AppController
…
104        $this->request->data['Comment']['text'] = Sanitize::html($this->request-
>data['Comment']['text']);
105        if (($auth & COMMENT_AUTH_NAME) == 0) {
```

Code snippet to sanitize Controller's GuestsController.php

Controller/GuestsController.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
15  * @license       GPL-2.0 (http://www.opensource.org/licenses/GPL-2.0)
16  */
17
18 App::uses('Sanitize', 'Utility');
19
20 class GuestsController extends AppController {
…
61     $userId = $this->getUserId();
62     $escName = Sanitize::escape($this->request->data['Group']['name']);
```



Security Test	OWASP Category	Yes/No	Results
Testing for SQL Injection	OTG-INPVAL-005	PASS	Application is secure from SQL injection

Explanation:
A successful SQL injection attack can read sensitive data from the database, modify database data (insert/update/delete), execute administration operations on the database (such as shutdown the DBMS), recover the content of a given file existing on the DBMS file system or write files into the file system or even log in to the system.

We ran couple of tests on the input text fields present on phTagr and this security test has PASSED.

Controller/UsersController.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
135         $this->Session->setFlash(__("Sorry. Your account is expired!"));
136       } else {
137         $user = $this->User->decrypt($user);
138         if ($user['User']['password'] == $this->request->data['User']['password']) {
...
391             $this->request->data['User']['username'], $this->request->data['User']
    ['email']));
392       } else {
393         $user = $this->User->decrypt($user);
```

Console/Command/PasswdShell.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
34      $this->help();
35    }
36
37    /** Decrypt all passwords */
38    function encrypt() {
39      if ('y' == $this->in('Are you sure you want to upgrade all passwords?', array('y',
    'n'), 'n')) {
…
47            $this->error("Database Error", "Could not save encrypted password");
48          }
49          $this->out("Encrypt password of user '{$user['User']['username']}'");
```

Vendor/Pear/Crypt/Blowfish.php PHP

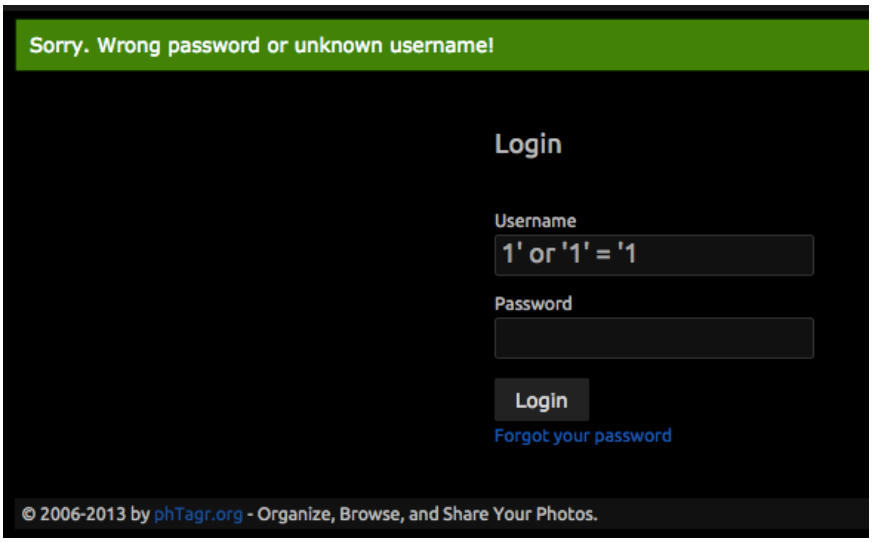
Showing the top two matches Last indexed on Sep 14, 2016

```
36    * $encrypted = $bf->encrypt('this is some example plain text');
37    * $plaintext = $bf->decrypt($encrypted);
38    * echo "plain text: $plaintext";
…
189     * @return string Returns cipher text on success, PEAR_Error on failure
190     * @access public
191     */
192    function encrypt($plainText)
```

Config/core.php PHP

Showing the top match Last indexed on Sep 14, 2016

```
269   * A random numeric string (digits only) used to encrypt/decrypt strings.
270   */
271         Configure::write('Security.cipherSeed', '76859309657453542496749683645');
```



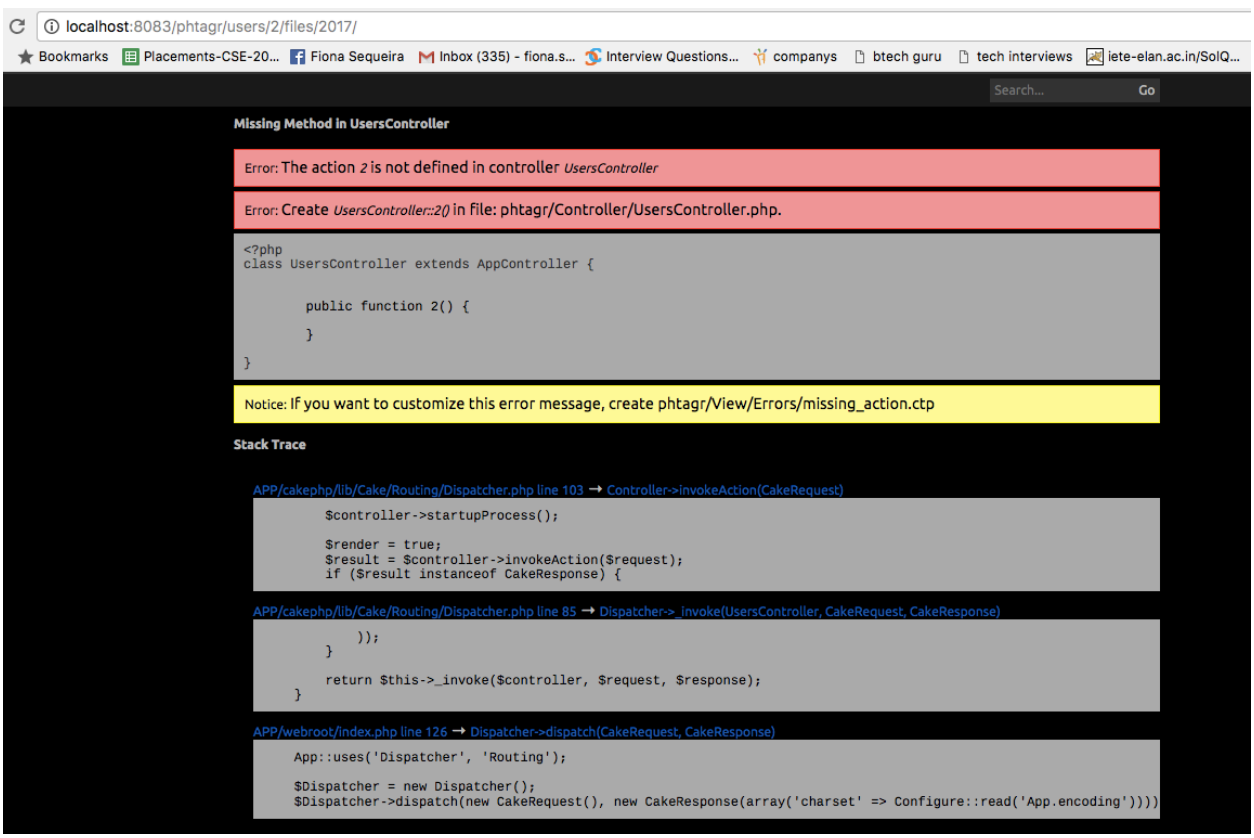
Security Test	OWASP Category	Yes/No	Results
Testing for Local File Inclusion	**OTG-INPVAL-012**	**PASS**	**Local file inclusion is protected.**

Explanation:

In the File Inclusion potential vulnerability, would allow an attacker to include a file, usually exploiting a "dynamic file inclusion" mechanisms possibly through the URL using an example like this

Localhost:8083/phtagr/setup/../../../etc/passwd

and implemented in the target application. The vulnerability occurs due to the use of user-supplied input without proper validation. The application is secure for local file inclusion as the url provides multiple redirections and doesn't give the direct reference of folders in use. Hence this security test has passed.



TESTING FOR ERROR HANDLING

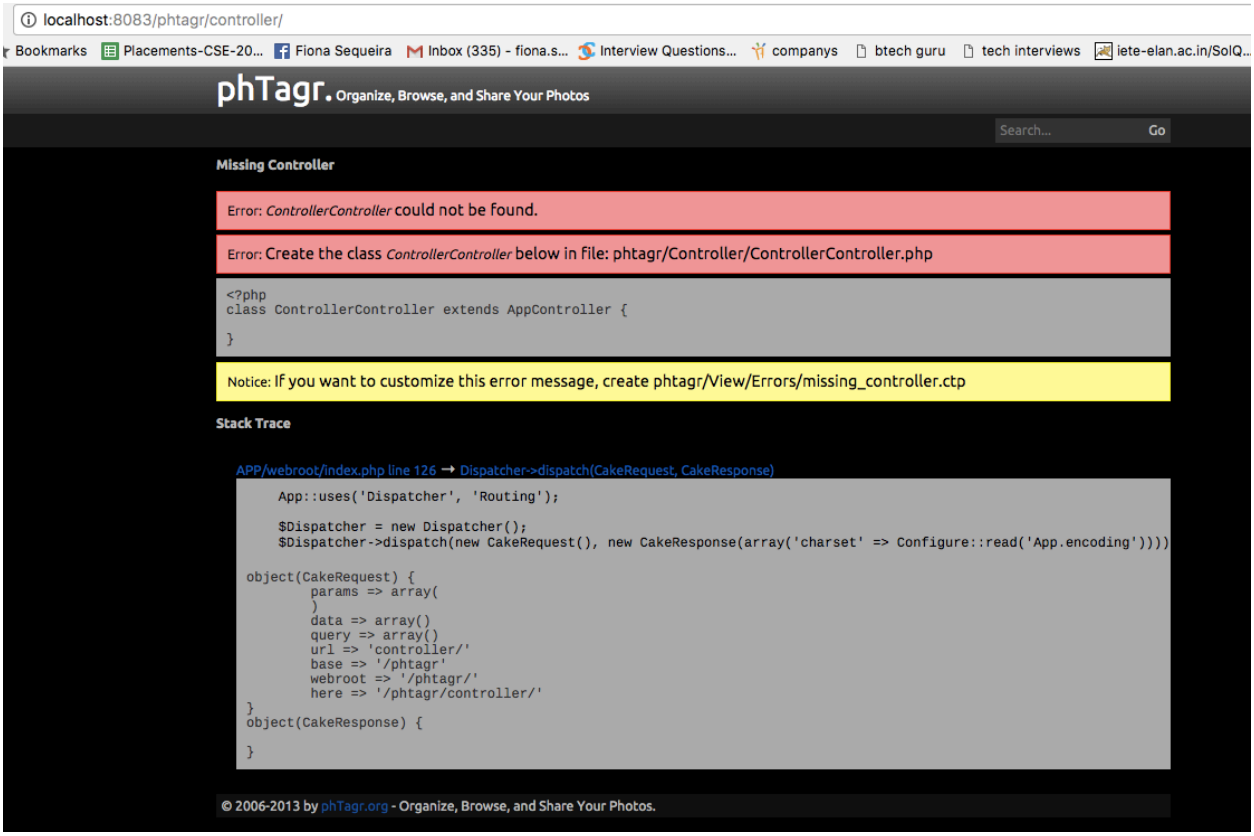
Security Test	OWASP Category	Yes/No	Results
Analysis of Stack trace	OTG-SESS-002	FAIL	Stack traces available at user's discretion.

Explanation:

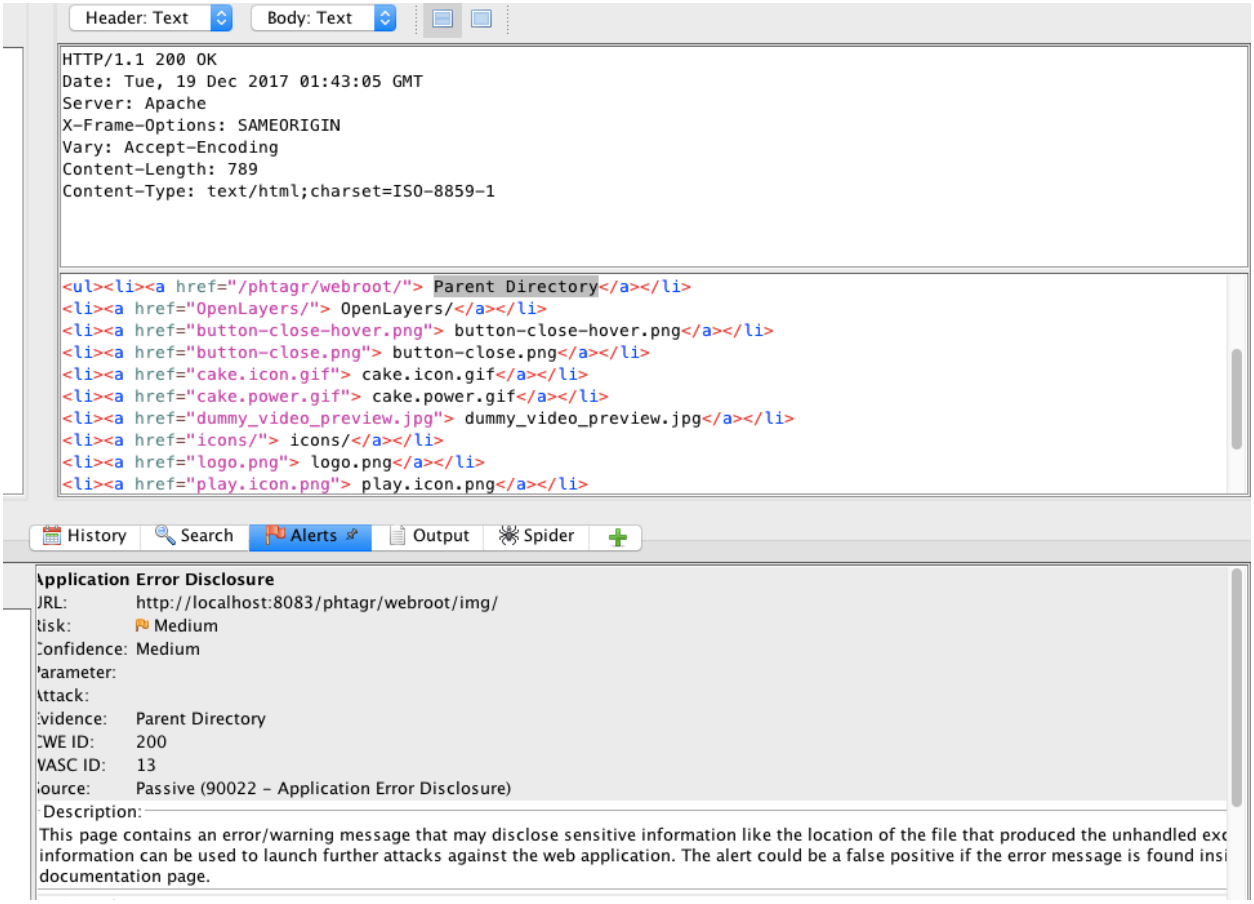
When the application responds with stack traces that are not managed it could reveal information useful to attackers. Providing debugging information like the below screenshot as a result of operations that generate errors is considered a bad practice due to multiple reasons. This contains information on internal workings of the application such as relative paths of the point where the application is installed and how objects are referenced internally.

Hence this security test is FAILED.

Suggested Remedy: Stack traces tend to let out essential information about the internal working of an application and relative paths where the error occurrences and the way objects are referenced, which can be really dangerous. Stack traces should not be displayed as the user interface.



Following is the ZAP tool analysis



TESTING FOR WEAK CRYPTOGRAPHY

Security Test	OWASP Category	Yes/No	Results
Testing for Weak SSL/TLS Ciphers, Insufficient Transport Layer Protection	OTG-CRYPST-001	Not Applicable	Application is run using HTTP not HTTPS

Security Test	OWASP Category	Yes/No	Results
Testing for Padding Oracle	OTG-CRYPST-002	Not Applicable	Application is run using HTTP not HTTPS

Security Test	OWASP Category	Yes/No	Results
Testing for Sensitive information sent via unencrypted channels	OTG-CRYPST-003	Not Applicable	Application is run using HTTP not HTTPS

Security Test	OWASP Category	Yes/No	Results
Testing for Weak Encryption	OTG-CRYPST-004	Not Applicable	Application is run using HTTP not HTTPS

Business Logic Testing

Security Test	OWASP Category	Yes/No	Results
Test Upload of Unexpected File Types	OTG-BUSLOGIC-008	FAIL	Successful upload of Unexpected File types to the database and uploaded to the local folder.

Explanation:
Vulnerabilities related to the upload of unexpected file types is unique in that the upload should quickly reject a file if it does not have a specific extension. Additionally, this is different from uploading malicious files in that in most cases an incorrect file format may not by itself be inherently "malicious" but may be detrimental to the saved data. For example, if an application accepts Windows Excel files, if an similar database file is uploaded it may be read but data extracted may be moved to incorrect locations. Successful upload of .exe and .pdf files were noticed without any error message. This security test has **FAILED**.

Remedy: The mime content of the uploaded file must be checked before proceeding during file upload. If it is of an invalid type, then it should throw an error to the user and the file upload should be halted.

```
SELECT * FROM `files`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refr

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Options

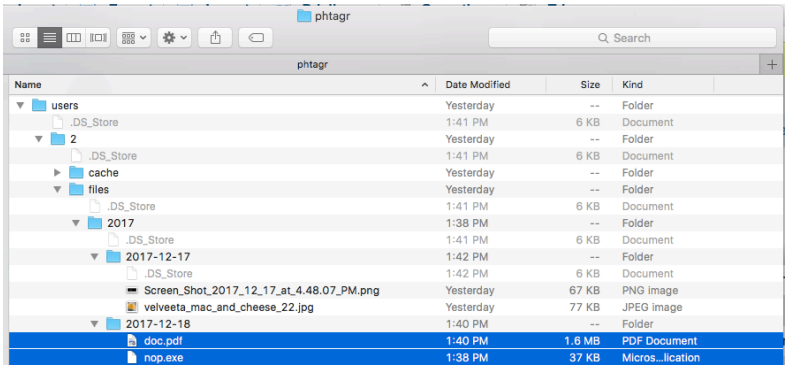
	id	created	modified	user_id	media_id	flag	type	path	file
Edit Copy Delete	3	2017-12-17 16:17:50	2017-12-17 16:17:50	2	1	4	3	/Users/Fiona/Desktop/phtagr5/apache2/htdocs/phtagr...	Screen_Shot
Edit Copy Delete	6	2017-12-18 10:40:52	2017-12-18 10:40:52	2	NULL	0	0	/Users/Fiona/Desktop/phtagr5/apache2/htdocs/phtagr...	doc.pdf

Model/MyFile.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
 * @return Type of the file. If the type is not known it returns
 * FILE_TYPE_UNKNOWN
 */
public function _getTypeFromFilename($filename) {
  $ext = strtolower(substr($filename, strrpos($filename, '.') + 1));
...
  foreach ($this->types as $type => $extensions) {
    if (in_array($ext, $extensions)) {
      return $type;
```

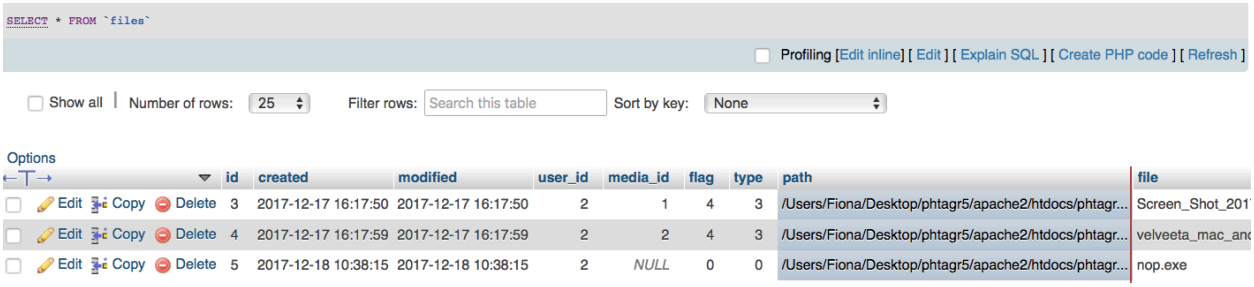
Screenshot of document saved on drive.

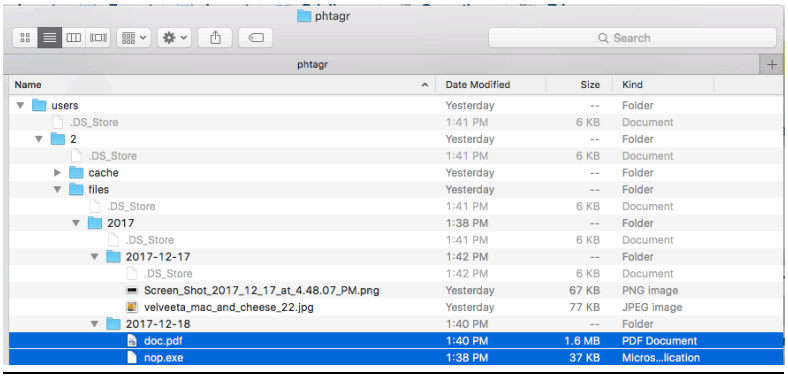


Security Test	OWASP Category	Yes/No	Results
Test Upload of Malicious File	OTG-BUSLOGIC-009	FAIL	Successful upload of File with .exe extension, which could potentially be malicious

Explanation:
We have made sure to check for the regular vulnerabilities like the validity and security of text but accepting files can introduce even more risk. To reduce the risk we may only accept certain file extensions, but attackers are able to encapsulate malicious code into inert file types. Testing for malicious files verifies that the application/system is able to correctly protect against attackers uploading malicious files. In this security test we have tried to upload a file type of .exe which potentially could be a virus file. Also we have verified that this file is saved on the system and thus the virus may spread itself or through remote processes exes or shell code can be executed for a successful PHP shell upload.

Remedy: The mime content of the uploaded file must be checked before proceeding during file upload. If it is of an invalid type then it should throw an error to the user. A whitelist of approved file types should be maintained and any other file type should be halted from being uploaded by the user.





SECURITY FEATURES ON THE APP

Cipher behavior
The cipher behavior of phtagr handles the symmetric encryption and decryption on the fly. For the symmetric encryption, the cipher BlowFish is used, but other symmetric encryption algorithms are possible. This behavior is very useful if your data should not be stored in clear text in the database (e.g. external database connections).The behavior could be configured simple for your needs

Model Properties
By default, the cipher behavior encrypts and decrypts the model property (table column) password. Other fields are also possible.

Model:

```
class User extends AppModel
{
  var $name = 'User';

  var $actsAs = array('Cipher' => array('cipher' => array('password', 'email', 'creditnumber')));
}
```

Custom Key
By default, the cipher behavior uses the Security.salt as cipher key. If you require a custom key, you can set in on the configuration:

Salt and Padding
Before a value is encrypted it will be packed and padded. The clear text before the value is ciphered is surrounded by a salt and padded to a specific length block to $salt.$value.$padding.$salt.

The salt is used to avoid same encrypted results of same values. It is also used to discover the correct decryption. The padding is used to hide the original value lengths. By default, the salt and padding have the length of 4. This could be changed in the behavior configuration.

Source Code Assessment

Issue #1
//TODO statements are present in multiple files

Having a //TODO present in the source code allows hackers and attackers to understand the functionalities that have not been implemented in the system as of now and find a way to circumvent the application via a potential opening made clearer with the //TODO item clause. These were found at multiple places within the source code, as the below screenshot.

View/Groups/dashboard_search.ctp PHP

Showing the top match Last indexed on Sep 14, 2016

```
5   <h2><?php echo __('Results') ?></h2>
6   <ul>
7   <?php if(!empty($groups)) {
8           // @TODO Put groups into a table where we also have a button to join the group
```

Controller/WebdavController.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
27    var $uses = array('User', 'MyFile', 'Media', 'Property', 'Lock');
28    // Important to set the davroot in the Webdav Server
29    var $name = 'webdav';
30
31    /** @todo Remove configuration of debug */
...
52      // Preload WebdavServer component which requires a running session
53      $this->loadComponent('WebdavServer');
54    }
55
56    /** @todo Set webdav root to creator's root if user is guest */
```

Vendor/Pear/HTTP/WebDAV/Server/Filesystem.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
114         // set root directory, defaults to webserver document root if not set
115         if ($base) {
116             $this->base = realpath($base); // TODO throw if not a directory
...
123         mysql_select_db($this->db_name) or die(mysql_error());
124         // TODO throw on connection problems
125
126         // let the base class do all the work
```

Controller/Component/WebdavServerComponent.php PHP

Showing the top two matches Last indexed on Sep 14, 2016

```
262     @return True if user is authorized to read directory. False otherwise
263     @todo This function must be implemented */
264   public function _canRead($fspath) {
265     if ($this->controller->getUserRole() >= ROLE_USER) {
...
306     // information for contained resources requested?
307     if (!empty($options["depth"])) { // TODO check for is_dir() first?
```

Issue #2
Cookie No HttpOnly Flag

Using the OWASP ZAP 2.7.0 tool we have noticed a cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.

```
HTTP/1.1 200 OK
Date: Tue, 19 Dec 2017 02:59:22 GMT
Server: Apache
X-Frame-Options: SAMEORIGIN
X-Powered-By: PHP/5.6.32
Set-Cookie: CAKEPHP=up5vcviqhd2iq499ospasuep62; expires=Tue, 19-Dec-2017 06:59:22 GMT; Max-Age=14400; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 4449
Vary: Accept-Encoding
```

Issue #3

Password Autocomplete in Browser

The AUTOCOMPLETE attribute is not disabled on an HTML FORM/INPUT element containing password type input. Passwords may be stored in browsers and retrieved. Below is a snapshot from the OWASP ZAP security scan

```
HTTP/1.1 200 OK
Date: Tue, 19 Dec 2017 02:59:23 GMT
Server: Apache
X-Frame-Options: SAMEORIGIN
X-Powered-By: PHP/5.6.32
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 3441
Vary: Accept-Encoding
Content-Type: text/html; charset=UTF-8
```

```
<div id="main"><div class="sub">

<div id="content" class="content content-users content-users-login"><div class="sub">

<form action="/phtagr/users/login" class="login" id="UserLoginForm" method="post" accept-charset="utf-8"><div
style="display:none;"><input type="hidden" name="_method" value="POST"/></div><fieldset>
<legend>Login</legend>
<div class="input text required"><label for="UserUsername">Username</label><input name="data[User][username]"
maxlength="32" type="text" id="UserUsername"/></div><div class="input password required"><label for=
"UserPassword">Password</label><input name="data[User][password]" type="password" id="UserPassword"/></div></
```

Issue #4

Web Browser XSS Protection Not Enabled

OWASP ZAP tool indicates Web Browser XSS Protection is not enabled, or is disabled by the configuration of the 'X-XSS-Protection' HTTP response header on the web server. The X-XSS-Protection HTTP response header allows the web server to enable or disable the web browser's XSS protection mechanism. The following values would attempt to enable it:

X-XSS-Protection: 1; mode=block

```
HTTP/1.1 200 OK
Date: Tue, 19 Dec 2017 02:59:22 GMT
Server: Apache
X-Frame-Options: SAMEORIGIN
X-Powered-By: PHP/5.6.32
Set-Cookie: CAKEPHP=up5vcviqhd2iq499ospasuep62; expires=Tue, 19-Dec-2017 06:59:22 GMT; Max-Age=14400; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 4449
Vary: Accept-Encoding
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">

<head>
<title>Home</title>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<link href="/phtagr/favicon.ico" type="image/x-icon" rel="icon" /><link href="/phtagr/favicon.ico" type=
"image/x-icon" rel="shortcut icon" />
<link rel="stylesheet" type="text/css" href="/phtagr/ccss/default.css" />
<script type="text/javascript" src="/phtagr/js/jquery-1.5.1.min.js"></script><link rel="stylesheet" type=
"text/css" href="/phtagr/ccss/custom-phtagr/jquery-ui-1.8.14.custom.css" /><script type="text/javascript" src
```

Issue#5

X-Content-Type-Options Header Missing – Low Risk

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

```
HTTP/1.1 200 OK
Date: Tue, 19 Dec 2017 02:59:22 GMT
Server: Apache
X-Frame-Options: SAMEORIGIN
X-Powered-By: PHP/5.6.32
Set-Cookie: CAKEPHP=up5vcviqhd2iq499ospasuep62; expires=Tue, 19-Dec-2017 06:59:22 GMT; Max-Age=14400; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Content-Length: 4449
Vary: Accept-Encoding
Content-Type: text/html; charset=UTF-8
```

Notes for Future Assessment

Code includes several locations where comments do not hide unimplemented functionalities and bugs. This is a possible attack surface.

Conclusion

After completing our assessment, we can conclude that the robustness of the application is quite average. The web framework is successfully able to block some of the most substantial web attacks. However, there were quite a few misses for many vulnerabilities. The setup instructions were quite extensive and it is easy to skip a few instructions, which allows the application to be easily misconfigured and thus deployed. The phTagr services could then still be vulnerable to a lot the above mentioned attacks.

References:

http://www.owasp.org/index.php
http://www.phtagr.org/download/
http://php.net/manual/en/index.php
http://trac.phtagr.org
https://en.wikipedia.org/wiki/Model–view–controller
https://www.owasp.org/index.php/PHP_Security_Cheat_Sheet
http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet
https://www.owasp.org/index.php/XSS_(Cross_Site_Scripting)_Prevention_Cheat_Sheet
https://blog.veracode.com/2014/03/guidelines-for-setting-security-headers/